

Raport de Securitate: Proiect DASS - Build, Hack & Secure

Alexandru Agusoaei

Aprilie 2026

Cuprins

1	Introducere	3
1.1	Contextul și Descrierea Aplicației	3
1.2	Roluri și Obiective	3
1.3	Arhitectura și Tehnologiile Utilizate	3
2	Setup Mediu și Inițializarea Proiectului	4
2.1	Configurarea Mașinii Virtuale	4
2.2	Instalarea Framework-ului (Next.js)	4
2.3	Configurarea Bazei de Date (Supabase)	5
3	Implementare MVP (Minimum Viable Product)	5
3.1	Structura Bazei de Date (Data Layer)	5
3.2	Modulul de Autentificare (Auth)	6
3.3	Modulele de Business (CRUD și Search)	6
4	Prezentare Vulnerabilități (Mapping OWASP)	7
4.1	4.1 Password Policy Slab (OWASP A07:2021 - Identification and Authentication Failures)	7
4.2	4.2 Stocare Nesigură a Parolelor (OWASP A02:2021 - Cryptographic Failures)	7
4.3	4.3 Brute Force / Lipsă Rate Limiting (OWASP A07:2021 - Identification and Authentication Failures)	8
4.4	4.4 User Enumeration (OWASP A07:2021 - Identification and Authentication Failures)	8
4.5	4.5 Gestionare Nesigură a Sesiunilor (OWASP A05:2021 - Security Misconfiguration)	8
4.6	4.6 Resetare Parolă Nesigură (OWASP A01:2021 - Broken Access Control)	8
5	Demonstrare Atac (Proof of Concept)	8
5.1	Atac 1: Exploatarea politicilor slabe și citirea parolelor în clar (Vuln 4.1 & 4.2)	8
5.2	Atac 2: Enumerarea Utilizatorilor (Vuln 4.4)	9
5.3	Atac 3: Brute Force prin lipsa Rate Limiting (Vuln 4.3)	10
5.4	Atac 4: Furt de sesiune prin cookie-uri nesecurizate (Vuln 4.5)	11
5.5	Atac 5: Compromiterea contului prin Resetare Predictibilă (Vuln 4.6)	12

6	Analiză Impact	13
7	Implementare Fix (Securizarea Aplicației - v2)	14
7.1	Remediere 4.1 & 4.2: Password Policy și Hashing Sigur	14
7.2	Remediere 4.3: Implementare Rate Limiting (Protecție Brute Force)	15
7.3	Remediere 4.4: Prevenirea Enumerării Utilizatorilor	15
7.4	Remediere 4.5: Securizarea Sesiunilor (Hardening Cookie-uri)	15
7.5	Remediere 4.6: Resetare Sigură a Parolei (Implementare JWT)	16
8	Re-test (Validarea Securității)	16
8.1	Validare 4.1 & 4.2: Password Policy și Hashing	16
8.2	Validare 4.4: User Enumeration	16
8.3	Validare 4.3: Protecție Brute Force (Rate Limiting)	17
8.4	Validare 4.5: Securitatea Sesiunilor	18
8.5	Validare 4.6: Resetarea Sigură a Parolei	18
9	Audit și Jurnalizare (Logging & Monitoring)	19
9.1	Trasabilitatea Evenimentelor de Securitate	19
9.2	Răspunsul la Incidente (Incident Response)	19
10	Concluzii și Lecții Învățate	20

1 Introducere

1.1 Contextul și Descrierea Aplicației

Prezentul raport documentează procesul de auditare și securizare a mecanismului de autentificare pentru o aplicație internă de login/register. Aplicația este dezvoltată de compania fictivă „AuthX” și este utilizată de către angajații acesteia pentru a accesa și gestiona resurse sensibile (tichete interne). Deși aplicația este deja folosită în mediul de producție, echipa de securitate a semnalat probleme grave legate de implementarea mecanismelor de autentificare și autorizare.

Scopul acestui proiect este înțelegerea modului în care un sistem de autentificare este atacat în practică și, mai important, cum trebuie implementat corect pentru a rezista atacurilor din lumea reală.

1.2 Roluri și Obiective

În cadrul acestui audit, am adoptat un dublu rol, urmărind fluxul de securitate SDLC (Build → Hack → Fix → Retest):

- **Security Tester:** Am identificat, exploatat și demonstrat vulnerabilitățile prezente în versiunea inițială a aplicației (v1), folosind tehnici de ethical hacking.
- **Developer:** Am remediat vulnerabilitățile identificate prin implementarea controalelor corecte de securitate (v2) și am validat fix-urile printr-un proces de re-testare.

Auditul s-a concentrat exclusiv pe componentele critice ale identității digitale, fără a pune accent pe funcționalități de business complexe. Ariile principale de investigare au fost:

- Logica de autentificare (Login);
- Gestionarea și stocarea parolelor;
- Managementul sesiunilor;
- Mecanismul de resetare a parolei (Forgot Password).

1.3 Arhitectura și Tehnologiile Utilizate

Pentru a construi un mediu realist și capabil să ruleze local conform cerințelor de livrare, arhitectura aplicației a fost dezvoltată folosind următoarele tehnologii moderne:

- **Next.js (App Router):** A fost utilizat ca framework full-stack. Partea de frontend (interfața utilizatorului) a fost construită folosind React și Tailwind CSS, în timp ce logica de backend a fost implementată prin intermediul API Routes direct în Next.js. Această abordare permite o comunicare rapidă și o gestionare facilă a cookie-urilor de sesiune.
- **Supabase (PostgreSQL):** Pentru stratul de persistență a datelor (Data Layer) am optat pentru Supabase, care oferă o bază de date PostgreSQL reală. Această alegere respectă recomandările academice pentru a susține exersarea injecțiilor SQL reale și permite stocarea sigură și structurată a utilizatorilor, tichetelor și logurilor de audit.

- **Mediul de virtualizare:** Toate testele, rularea aplicației și demonstrarea atacurilor au fost efectuate în interiorul unei mașini virtuale (VM) Linux, asigurând un mediu izolat și controlat (Lab Environment).

2 Setup Mediu și Inițializarea Proiectului

Pentru a asigura un mediu de testare izolat, controlat și reproductibil, conform cerințelor de securitate, proiectul a fost dezvoltat și auditat într-un mediu de laborator (Lab Environment).

2.1 Configurarea Mașinii Virtuale

Primul pas în realizarea proiectului a constat în configurarea mașinii virtuale (VM). Am optat pentru utilizarea unui sistem de operare bazat pe Linux (Ubuntu), rulat prin intermediul unui hypervisor (VirtualBox).

Pentru a asigura autenticitatea muncii, sistemul de operare a fost configurat cu un hostname specific proiectului și un utilizator personalizat, elemente vizibile în prompt-ul terminalului la fiecare execuție a comenzilor. (*alexandru-agusoei-VirtualBox*)

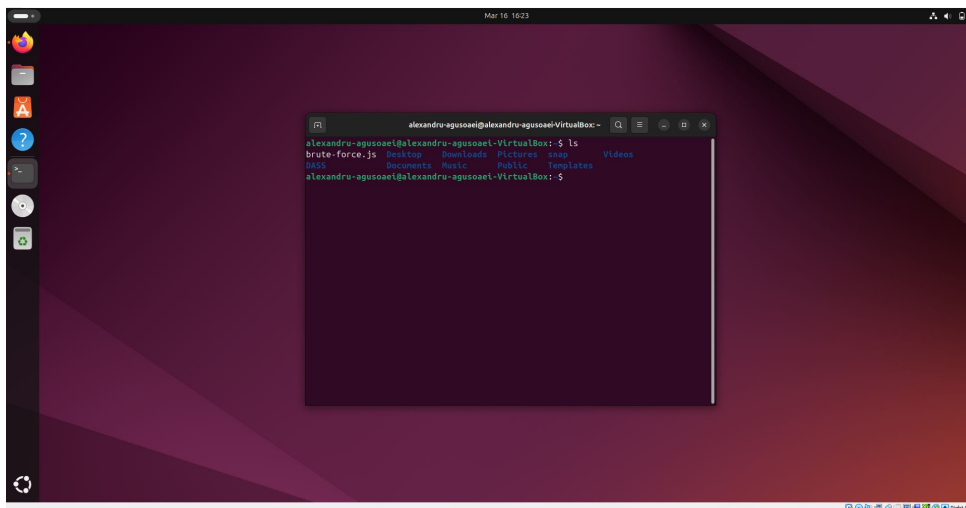


Figura 1: Interfața mașinii virtuale Ubuntu cu terminalul deschis, demonstrând prompt-ul cu username și hostname.

2.2 Instalarea Framework-ului (Next.js)

Dezvoltarea aplicației a necesitat instalarea mediului de execuție Node.js și a managerului de pachete npm pe mașina virtuală. Ulterior, am inițializat framework-ul Next.js folosind App Router, executând următoarea comandă în terminal:

```
npx create-next-app@latest dass
```

Această comandă a generat structura de bază a proiectului, incluzând suport pentru TypeScript, ESLint și Tailwind CSS pentru o stilizare rapidă a interfețelor grafice. Imediat după inițializare, am configurat un repository Git local și am creat branch-ul *vulnerable-v1* pentru a stoca implementarea inițială, nesecurizată.

2.3 Configurarea Bazei de Date (Supabase)

Deși mediul local suportă baze de date precum SQLite, am ales să integrez Supabase, un serviciu de tip Backend-as-a-Service (BaaS) bazat pe PostgreSQL, pentru a simula o arhitectură cloud modernă și realistă.

Integrarea bazei de date a presupus următorii pași:

1. Crearea proiectului în dashboard-ul Supabase și generarea cheilor de acces (URL și Anon Key).
2. Crearea tabelelor necesare (`users`, `tickets`, `audit_logs`) folosind scripturi SQL direct în consola bazei de date.
3. Conectarea aplicației Next.js la baza de date instalând clientul oficial: `npm install @supabase/supabase-js`.
4. Securizarea cheilor de API prin salvarea acestora într-un fișier local `.env.local`, urmată de adăugarea acestui fișier în `.gitignore` pentru a preveni expunerea accidentală a datelor sensibile pe repository-ul public.

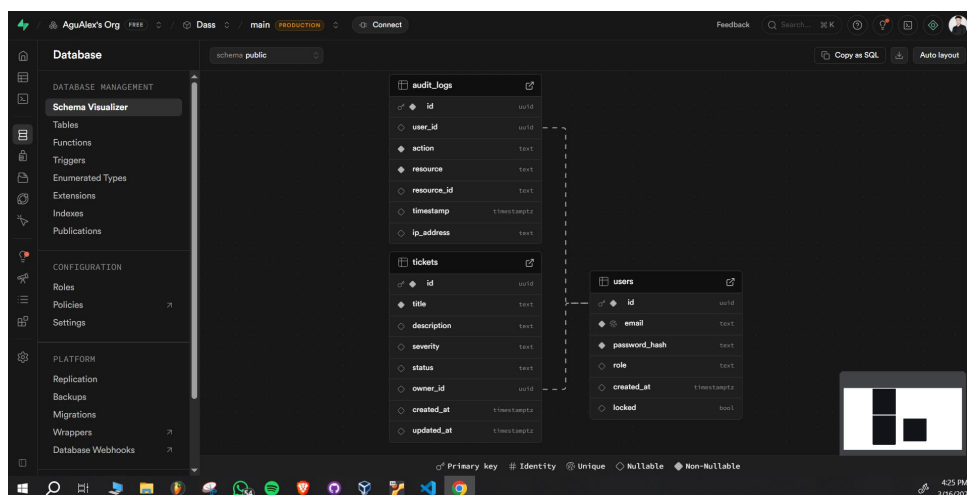


Figura 2: Structura bazei de date PostgreSQL (Supabase) configurată pentru proiect.

Prin finalizarea acestui setup, mediul de dezvoltare a devenit complet funcțional, permițând trecerea la implementarea logicilor de autentificare și autorizare vulnerabile.

3 Implementare MVP (Minimum Viable Product)

Pentru a putea testa și audita mecanismele de securitate, a fost necesară dezvoltarea unei versiuni inițiale complet funcționale a aplicației (MVP). Această versiune include gestionarea identității utilizatorilor, operațiuni de tip CRUD (Create, Read, Update, Delete) pentru resursele interne și o funcție de căutare.

3.1 Structura Bazei de Date (Data Layer)

Pentru a susține logica de business și mecanismele de autentificare, baza de date a fost structurată în trei tabele principale, implementate în Supabase (PostgreSQL), respectând un model de date realist la nivel mediu:

- **Tabelul users:** Stocază identitățile angajaților. Câmpurile principale includ `id` (UUID), `email` (folosit ca username), `password_hash` (pentru stocarea parolelor), `role` (ex: USER, MANAGER) și un indicator `locked` pentru gestionarea conturilor blocate.
- **Tabelul tickets:** Reprezintă resursa de business a aplicației. Conține detalii despre incidentele raportate: `id`, `title`, `description`, `severity` (LOW/MED/HIGH), `status` (OPEN/IN PROGRESS/RESOLVED) și `owner_id` (cheie externă către `users.id`, esențială ulterior pentru verificarea autorizării și prevenirea IDOR).
- **Tabelul audit_logs:** Asigură trasabilitatea acțiunilor. Înregistrează evenimente precum autentificările sau crearea de tichete, reținând `user_id`, `action`, `resource` și `ip_address`.

3.2 Modulul de Autentificare (Auth)

În versiunea inițială (v1), modulul de autentificare a fost implementat funcțional, dar intenționat lăsat vulnerabil pentru a permite demonstrarea atacurilor. Rutele de API (Backend) dezvoltate în Next.js cuprind:

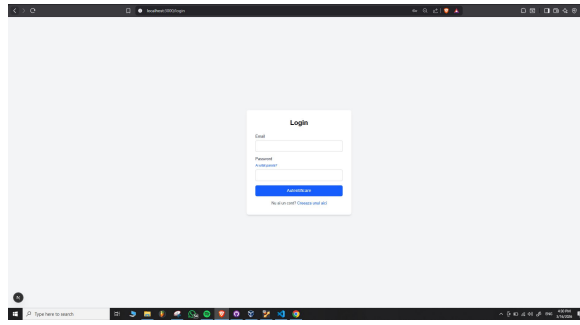
- **Register (/api/auth/register):** Preia datele din formular și inserează utilizatorul în baza de date. Input-ul este procesat în backend, deși în această versiune lipsește validarea complexității parolei.
- **Login (/api/auth/login):** Verifică existența utilizatorului și potrivirea credențialelor. La succes, generează o sesiune bazată pe cookie-uri.
- **Logout (/api/auth/logout):** Invalidează sesiunea prin ștergerea cookie-ului din browser-ul utilizatorului.
- **Forgot & Reset Password:** Un mecanism simplificat care generează un token predictibil și permite setarea unei noi parole.

3.3 Modulele de Business (CRUD și Search)

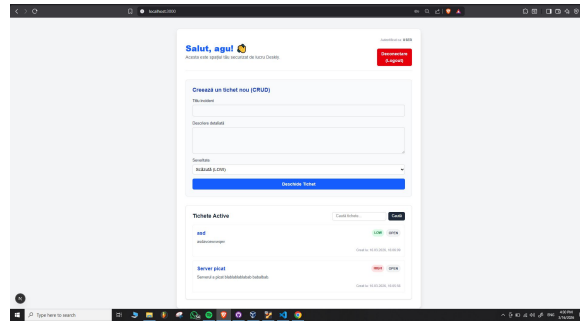
Aplicația permite utilizatorilor autentificați să interacționeze cu resursele interne (tichetele). Aceste module includ:

- **Creare și Vizualizare (CRUD):** Utilizatorii pot deschide tichete noi, specificând titlul, descrierea și severitatea. Acestea sunt salvate în baza de date, asociindu-se automat `owner_id`-ul utilizatorului curent pe baza sesiunii active.
- **Căutare și Filtrare (Search):** A fost implementat un endpoint care permite căutarea tichetelor după cuvinte cheie în titlu sau descriere, oferind o funcționalitate de bază pentru managementul incidentelor.

Această implementare MVP ne oferă suprafața de atac necesară (attack surface) pentru a începe auditul de securitate, întrucât logica de business depinde direct de robustețea mecanismului de autentificare.



(a) Formularul de autentificare



(b) Dashboard-ul principal

Figura 3: Interfața utilizatorului prezentând formularul de autentificare și dashboard-ul principal al aplicației.

4 Prezentare Vulnerabilități (Mapping OWASP)

În cadrul versiunii inițiale a aplicației (v1), au fost identificate și documentate o serie de vulnerabilități critice la nivelul mecanismului de autentificare și gestionare a sesiunilor. Aceste defecte au fost lăsate intenționat pentru a demonstra impactul ignorării bunelor practici de securitate și pot fi mapate direct pe standardul recunoscut internațional OWASP Top 10.

Mai jos sunt detaliate cele 6 vulnerabilități investigate:

4.1 4.1 Password Policy Slab (OWASP A07:2021 - Identification and Authentication Failures)

Descriere tehnică: Formularul de înregistrare (**Register**) nu implementează nicio validare a complexității sau lungimii parolei în backend. Sistemul acceptă parole foarte scurte sau triviale (ex: „123”, „password”), facilitând masiv atacurile de ghicire a parolelor.

4.2 4.2 Stocare Nesigură a Parolelor (OWASP A02:2021 - Cryptographic Failures)

Descriere tehnică: Parolele utilizatorilor sunt stocate în clar (plaintext) direct în baza de date, fără a utiliza funcții de derivare a cheilor (hashing) precum bcrypt sau Argon2. Aceasta reprezintă o încălcare critică a confidențialității: în cazul compromiterii bazei de date (simulată prin acces direct la DB), toate conturile sunt expuse imediat.

4.3 4.3 Brute Force / Lipsă Rate Limiting (OWASP A07:2021 - Identification and Authentication Failures)

Descriere tehnică: Endpoint-ul de Login permite un număr nelimitat de încercări de autentificare eșuate de la aceeași adresă IP. Lipsa unui mecanism de blocare a contului (account lockout) sau de limitare a ratei de request-uri (rate limiting) permite atacatorilor să folosească scripturi automatizate pentru atacuri de tip Brute Force sau Credential Stuffing.

4.4 4.4 User Enumeration (OWASP A07:2021 - Identification and Authentication Failures)

Descriere tehnică: La încercarea de autentificare, serverul răspunde cu mesaje de eroare distincte în funcție de starea contului: returnează „user inexistent” dacă adresa de email nu se află în baza de date, respectiv „parolă greșită” dacă adresa există. Această discrepanță permite unui atacator să enumere utilizatorii valizi ai platformei pe baza mesajelor primite.

4.5 4.5 Gestionare Nesigură a Sesiunilor (OWASP A05:2021 - Security Misconfiguration)

Descriere tehnică: După o autentificare reușită, aplicația emite un cookie de sesiune care nu este protejat prin flag-urile de securitate esențiale: `HttpOnly`, `Secure` și `SameSite`. De asemenea, token-ul generat are o durată de expirare prea lungă (ex: 1 an). Aceste configurări greșite expun aplicația la atacuri de tip Cross-Site Scripting (XSS) - permițând furtul token-ului prin JavaScript - și Cross-Site Request Forgery (CSRF).

4.6 4.6 Resetare Parolă Nesigură (OWASP A01:2021 - Broken Access Control)

Descriere tehnică: Funcționalitatea „Forgot Password” utilizează un token de resetare predictibil (derivat direct din encodarea Base64 a adresei de email). Token-ul este reutilizabil și nu are o limită de expirare. Această vulnerabilitate permite unui atacator să genereze manual link-ul de resetare pentru orice cont (inclusiv conturi de administratori) și să reseteze parola fără un control corect de autorizare.

5 Demonstrare Atac (Proof of Concept)

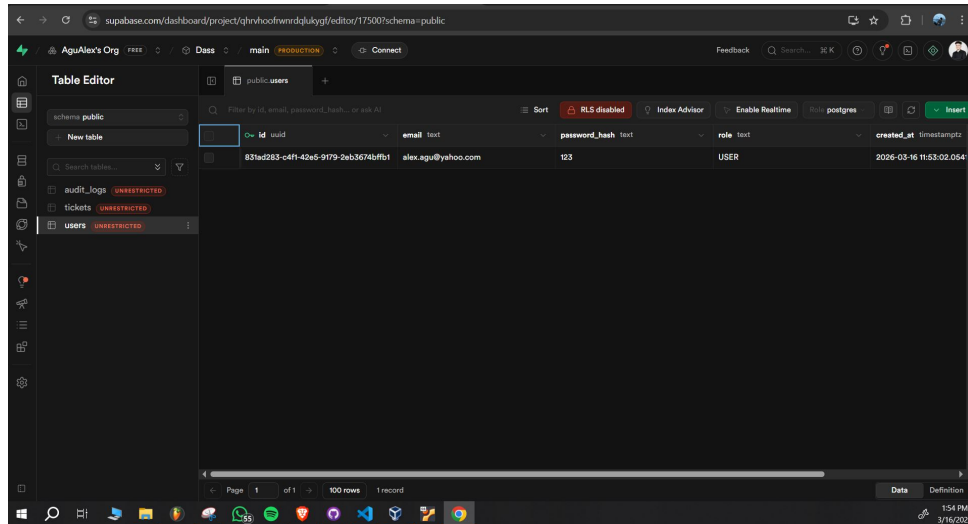
Această secțiune detaliază pașii de exploatare (PoC) pentru vulnerabilitățile identificate, demonstrând reproductibilitatea acestora într-un mediu controlat. Toate atacurile au fost executate din interiorul mașinii virtuale de test, dovezile incluzând identificatorii de sistem necesari (username, hostname, data și ora).

5.1 Atac 1: Exploatarea politicilor slabe și citirea parolelor în clar (Vuln 4.1 & 4.2)

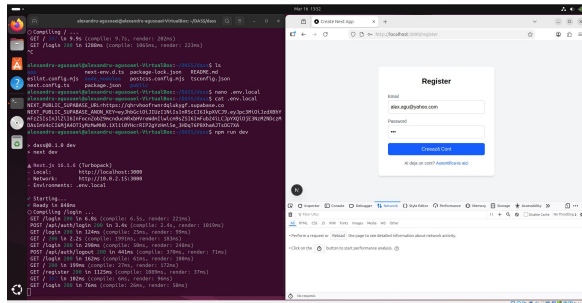
Pași de exploatare:

1. Am accesat endpoint-ul de înregistrare al aplicației (/register).
2. Am creat un cont utilizând o parolă trivială, formată din doar 3 caractere (ex: „123”), demonstrând lipsa validării la nivel de backend (Password Policy slab).
3. Am interogat direct baza de date (tabelul `users`) pentru a verifica modul de stocare.

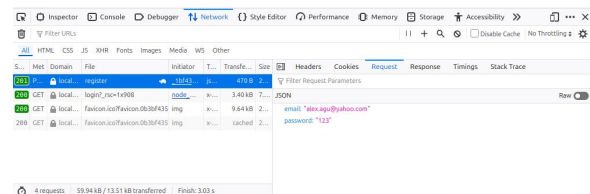
Rezultat: Parola a fost stocată exact sub forma „123”, confirmând absența oricărui mecanism de hashing (ex: bcrypt/argon2).



(a) Vizualizarea bazei de date demonstrând stocarea parolei în clar, fără aplicarea unui algoritm de hashing.



(b) Formularul de register arătând inputurile utilizatorului.



(c) Cererea HTTP interceptată conținând parola în clar.

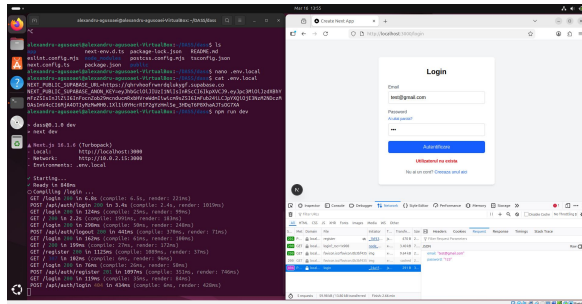
Figura 4: Demonstrarea stocării parolei în clar: (a) vizualizarea bazei de date, (b) formularul de autentificare, (c) interceptarea traficului.

5.2 Atac 2: Enumerarea Utilizatorilor (Vuln 4.4)

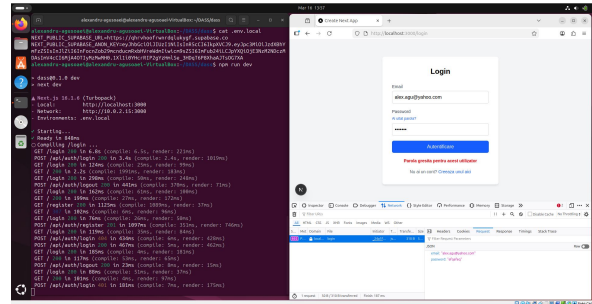
Pași de exploatare:

1. Am trimis un request de login (POST /api/auth/login) cu o adresă de email care nu există în sistem (ex: `test-fals@gmail.com`). Răspunsul serverului a fost: „*Utilizatorul nu există în sistem*”.
2. Am trimis un al doilea request cu o adresă de email validă, dar cu o parolă intenționat greșită. Răspunsul serverului s-a modificat în: „*Parolă greșită pentru acest utilizator*”.

Rezultat: Diferențierea mesajelor de eroare permite unui atacator să valideze ce adrese de email sunt înregistrate pe platformă.



(a) Răspuns pentru utilizator existent.



(b) Răspuns pentru utilizator inexistent.

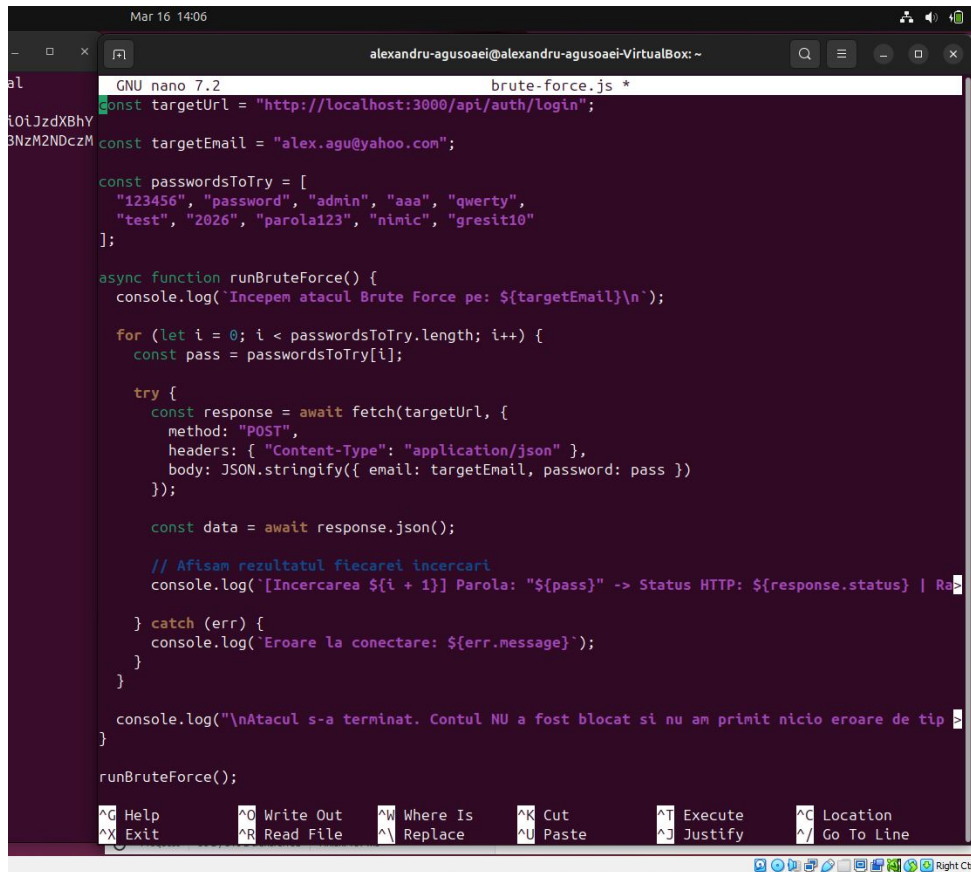
Figura 5: Răspunsuri HTTP distincte care confirmă vulnerabilitatea de User Enumeration.

5.3 Atac 3: Brute Force prin lipsa Rate Limiting (Vuln 4.3)

Pași de exploatare:

1. Am dezvoltat un script simplu în Node.js (**brute-force.js**) care trimite cereri HTTP POST succesive și rapide către endpoint-ul de login, iterând printr-un dicționar de parole.
2. Am rulat scriptul din terminalul mașinii virtuale împotriva contului valid identificat la pasul anterior.

Rezultat: Serverul a procesat toate cererile, returnând status 401 **Unauthorized** pentru fiecare, fără a bloca contul după N încercări eșuate sau a întoarce statusul 429 **Too Many Requests**.



```
GNU nano 7.2      brute-force.js *
const targetUrl = "http://localhost:3000/api/auth/login";
const targetEmail = "alex.agu@yahoo.com";

const passwordsToTry = [
  "123456", "password", "admin", "aaa", "qwerty",
  "test", "2026", "parola123", "ninic", "gresit10"
];

async function runBruteForce() {
  console.log(`Incepem atacul Brute Force pe: ${targetEmail}\n`);

  for (let i = 0; i < passwordsToTry.length; i++) {
    const pass = passwordsToTry[i];

    try {
      const response = await fetch(targetUrl, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ email: targetEmail, password: pass })
      });

      const data = await response.json();

      // Afisam rezultatul fiecarei incercari
      console.log(`[Incerarea ${i + 1}] Parola: "${pass}" -> Status HTTP: ${response.status} | Ra

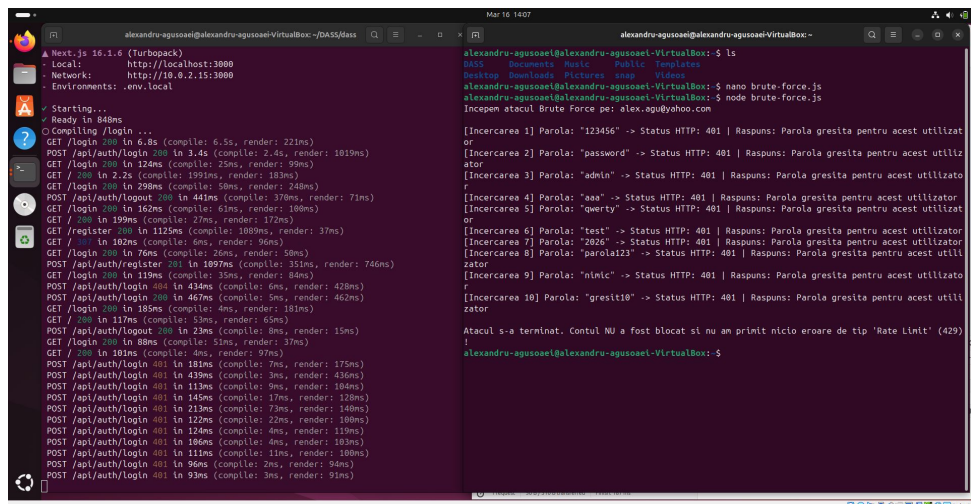
    } catch (err) {
      console.log(`Eroare la conectare: ${err.message}`);
    }
  }

  console.log("\nAtacul s-a terminat. Contul NU a fost blocat si nu am printat nicio eroare de tip >

runBruteForce();

Help      Write Out  Where Is  Cut       Execute   Location
Exit      Read File  Replace   Paste     Justify   Go To Line
```

Figura 6: Vizualizarea scriptului Node.js (brute-force.js) folosit pentru atacul de Brute Force.



```
alexandru-agusoei@alexandru-agusoei-VirtualBox: ~$ node brute-force.js
Incepem atacul Brute Force pe: alex.agu@yahoo.com

[Incerarea 1] Parola: "123456" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 2] Parola: "password" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 3] Parola: "admin" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 4] Parola: "aaa" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 5] Parola: "qwerty" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 6] Parola: "test" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 7] Parola: "2026" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 8] Parola: "parola123" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 9] Parola: "ninic" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator
[Incerarea 10] Parola: "gresit10" -> Status HTTP: 401 | Raspuns: Parola gresita pentru acest utilizator

Atacul s-a terminat. Contul NU a fost blocat si nu am printat nicio eroare de tip 'Rate Limit' (429)
alexandru-agusoei@alexandru-agusoei-VirtualBox: ~$
```

Figura 7: Executia scriptului de Brute Force; se observă lipsa blocării request-urilor multiple.

5.4 Atac 4: Furt de sesiune prin cookie-uri nesecurizate (Vuln 4.5)

Pași de exploatare:

1. Am efectuat o autentificare validă în aplicație.
2. Am deschis consola dezvoltatorului (Developer Tools) din browser, navigând la secțiunea *Application* → *Cookies*.

Rezultat: S-a constatat că token-ul de sesiune (`session.token`) este setat fără flag-urile obligatorii `HttpOnly` și `Secure`. Lipsa flag-ului `HttpOnly` face cookie-ul accesibil prin JavaScript (`document.cookie`), facilitând furtul acestuia în cazul unui atac Cross-Site Scripting (XSS).

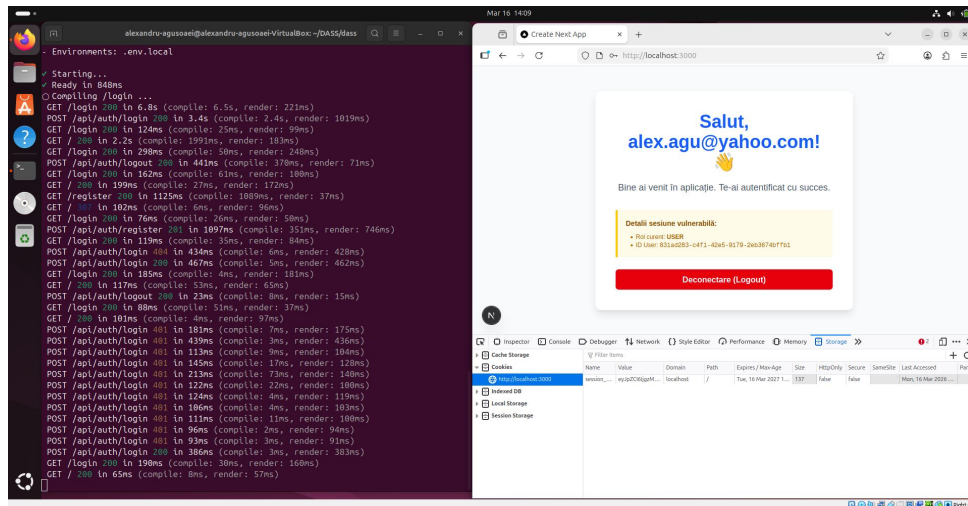


Figura 8: Inspectia cookie-ului de sesiune evidențiind absența flag-urilor de securitate.

5.5 Atac 5: Compromiterea contului prin Resetare Predictibilă (Vuln 4.6)

Pași de exploatare:

1. Am identificat contul victimei (ex: `alex.agu@yahoo.com`).
2. Cunoscând logica vulnerabilă a aplicației, am encodat adresa de email în Base64 folosind terminalul Linux: `echo -n "alex.agu@yahoo.com" | base64`, rezultând token-ul predictibil.
3. Am accesat direct URL-ul de resetare: `/reset-password?token=[TOKEN_BASE64]`.
4. Am introdus o nouă parolă.

Rezultat: Parola a fost schimbată cu succes fără a avea acces la căsuța de email a victimei, demonstrând compromiterea totală a contului (Account Takeover).

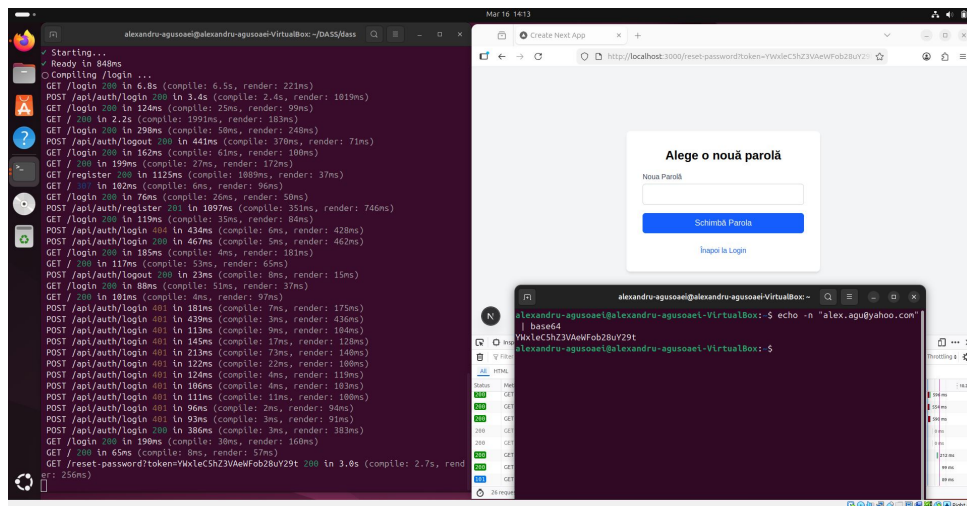


Figura 9: Generarea manuală a token-ului predictibil și bypass-ul mecanismului de resetare a parolei.

6 Analiză Impact

Exploatarea vulnerabilităților identificate în versiunea inițială a aplicației are un impact critic asupra securității organizației AuthX, compromițând principiile fundamentale ale securității informației: Confidențialitatea, Integritatea și Disponibilitatea (Triada CIA).

Prin înlănțuirea acestor vulnerabilități, un atacator poate obține următoarele:

- **Compromiterea Totală a Conturilor (Account Takeover - ATO):** Folosind mecanismul nesigur de resetare a parolei (token predictibil) sau atacuri de tip Brute Force (din cauza lipsei de rate limiting), un atacator poate prelua controlul oricărui cont de utilizator. Dacă este vizat un cont cu rol de **MANAGER**, atacatorul obține privilegii extinse.
- **Expunerea Datelor Sensibile (Data Breach):** Stocarea parolelor în clar în baza de date înseamnă că o singură breșă la nivel de rețea sau o injecție SQL reușită va expune credențialele tuturor angajaților. Deoarece utilizatorii au tendința de a refolosi parolele, acest lucru deschide calea către atacuri de tip *Credential Stuffing* pe alte platforme folosite de aceștia.
- **Acces Neautorizat la Resursele de Business:** Furtul de sesiune (facilitat de cookie-urile nesecurizate) permite unui atacator să bypass-eze complet pașii de autentificare. Odată intrat în sistem, acesta poate vizualiza, modifica sau șterge tichete interne sensibile, putând altera statusul incidentelor sau descrierile acestora, afectând grav integritatea datelor.
- **Facilitarea Atacurilor Țintite (Reconnaissance):** Vulnerabilitatea de User Enumeration îi oferă atacatorului o listă validă și confirmată a adreselor de email folosite în companie, listă ce poate fi utilizată ulterior pentru campanii de Phishing foarte bine direcționate (Spear Phishing).

În concluzie, implementarea deficitară a mecanismului de autentificare transformă aplicația dintr-un instrument intern util într-un risc major de securitate pentru compania

AuthX, putând duce la scurgeri de informații confidențiale, pierderi financiare și afectarea reputației.

7 Implementare Fix (Securizarea Aplicației - v2)

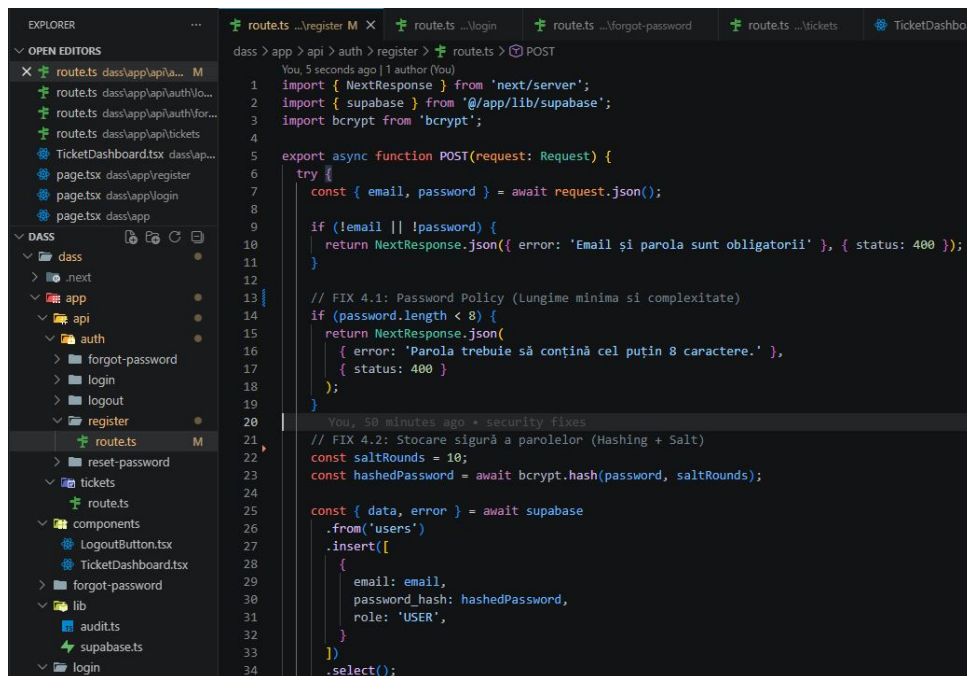
După finalizarea auditului de securitate pe versiunea `vulnerable-v1`, am inițiat un branch nou (`secure-v2`) pentru a remedia toate defectele identificate, respectând principiul *Defense in Depth* și recomandările OWASP.

Mai jos sunt detaliate soluțiile tehnice implementate pentru fiecare vulnerabilitate în parte:

7.1 Remediere 4.1 & 4.2: Password Policy și Hashing Sigur

Pentru a preveni stocarea parolelor în clar și ghicirea ușoară a acestora, am modificat endpoint-ul de `/api/auth/register`:

- **Validare Backend:** Am introdus o verificare strictă a lungimii parolei (minimum 8 caractere).
- **Hashing cu bcrypt:** Am integrat biblioteca `bcrypt` (standard în industrie). Parola nu mai ajunge niciodată în clar în baza de date; în schimb, este generat un *hash* folosind un *salt* aleator și un *cost factor* de 10. Astfel, chiar dacă baza de date este compromisă, parolele sunt protejate împotriva atacurilor de tip Rainbow Table.



```
EXPLORER
  OPEN EDITORS
    routets.dass/app/api/auth/register M
    routets.dass/app/api/auth/login
    routets.dass/app/api/auth/forgot-password
    routets.dass/app/api/tickets
    TicketDashboard.tsx
    page.tsx
    page.tsx
    page.tsx
  DASS
    .next
    app
      api
        auth
        forgot-password
        login
        logout
        register
      routets
      reset-password
      tickets
    components
      LogoutButton.tsx
      TicketDashboard.tsx
    forgot-password
    lib
      audit.ts
      supabase.ts
      login
  routets ...register M X
  routets ...login
  routets ...forgot-password
  routets ...tickets
  TicketDashboard

dass > app > api > auth > register > routets > POST
You, 5 seconds ago | 1 author (You)
1 import { NextResponse } from 'next/server';
2 import { supabase } from '@app/lib/supabase';
3 import bcrypt from 'bcrypt';
4
5 export async function POST(request: Request) {
6   try {
7     const { email, password } = await request.json();
8
9     if (!email || !password) {
10       return NextResponse.json({ error: 'Email și parola sunt obligatorii' }, { status: 400 });
11     }
12
13     // FIX 4.1: Password Policy (Lungime minima si complexitate)
14     if (password.length < 8) {
15       return NextResponse.json(
16         { error: 'Parola trebuie să conțină cel puțin 8 caractere.' },
17         { status: 400 }
18       );
19     }
20
21     You, 50 minutes ago + security fixes
22     // FIX 4.2: Stocare sigură a parolelor (Hashing + Salt)
23     const saltRounds = 10;
24     const hashedPassword = await bcrypt.hash(password, saltRounds);
25
26     const { data, error } = await supabase
27       .from('users')
28       .insert([
29         {
30           email: email,
31           password_hash: hashedPassword,
32           role: 'USER',
33         }
34       ])
35       .select();
```

Figura 10: Secvență de cod demonstrând implementarea algoritmului `bcrypt` și validarea lungimii parolei.

7.2 Remediere 4.3: Implementare Rate Limiting (Protecție Brute Force)

Pentru a opri atacurile automatizate asupra formularului de autentificare, am implementat un mecanism de *Rate Limiting* în memorie pe endpoint-ul `/api/auth/login`. Sistemul contorizează încercările eșuate pe baza adresei IP. Dacă un utilizator depășește 5 încercări într-un interval de 15 minute, serverul respinge automat cererile următoare, returnând statusul HTTP 429 Too Many Requests.

7.3 Remediere 4.4: Prevenirea Enumerării Utilizatorilor

Pentru a opri scurgerea de informații (Information Disclosure) la autentificare, am unificat mesajele de eroare. Indiferent dacă adresa de email nu există în baza de date sau parola este greșită, aplicația returnează acum un mesaj generic: „*Email sau parolă incorectă*”. Aceeași logică a fost aplicată și la funcționalitatea de *Forgot Password*, returnând un mesaj de succes fals pentru email-urile inexistente.

7.4 Remediere 4.5: Securizarea Sesiunilor (Hardening Cookie-uri)

Pentru a preveni furtul de sesiune (XSS) și atacurile CSRF, am rescris logica de emitere a cookie-ului `session.token`:

- Am activat flag-ul `HttpOnly`: `true`, interzicând accesul la cookie din codul JavaScript de pe client.
- Am setat `SameSite`: `'lax'` pentru a proteja împotriva request-urilor forjate de pe alte domenii.
- Timpul de expirare (`maxAge`) a fost redus de la 1 an la 2 ore, limitând fereastra de oportunitate a unui atacator.

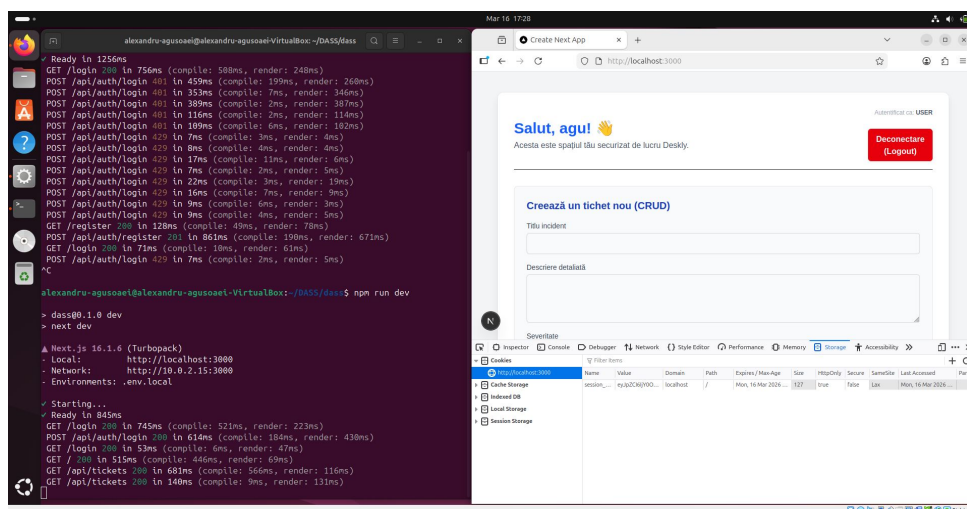


Figura 11: Configurarea strictă a flag-urilor de securitate pentru cookie-ul de sesiune.

7.5 Remediere 4.6: Resetare Sigură a Parolei (Implementare JWT)

Mecanismul predictibil bazat pe Base64 a fost complet eliminat și înlocuit cu **JSON Web Tokens (JWT)**. La solicitarea resetării, serverul generează acum un token semnat criptografic (*HMAC SHA-256*) utilizând o cheie secretă (`JWT_SECRET`). Mai mult, token-ului i s-a atribuit un termen de valabilitate strict (`expiresIn: '15m'`). Orice încercare de a falsifica token-ul sau de a-l folosi după expirare este respinsă automat de metoda `jwt.verify()`.

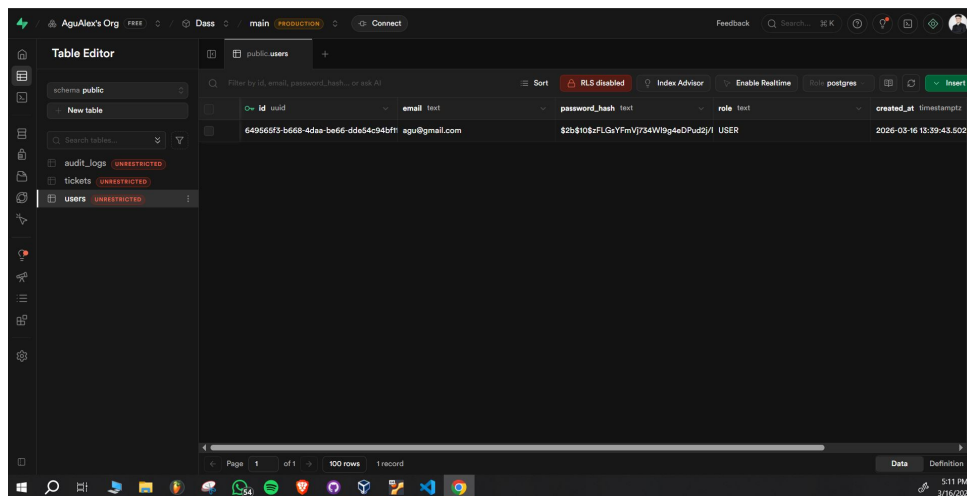
8 Re-test (Validarea Securității)

Faza de Re-test are ca scop validarea eficienței controalelor de securitate implementate în versiunea `secure-v2`. Toate atacurile detaliate în secțiunea de *Demonstrare Atac (PoC)* au fost reluate în aceleași condiții (pe mașina virtuală), vizând exact aceleași endpoint-uri.

8.1 Validare 4.1 & 4.2: Password Policy și Hashing

Acțiune: Am încercat crearea unui cont nou cu parola „123”. Ulterior, am creat un cont cu o parolă validă (minim 8 caractere) și am interogat baza de date.

Rezultat: Prima cerere a fost respinsă cu status `400 Bad Request` (validare lungime minimă). Pentru contul creat cu succes, baza de date afișează acum un hash `bcrypt` (ex: `$2b$10$F1GxYfmVj734W9p4eDPu2y/l`), demonstrând imposibilitatea citirii parolelor în clar.

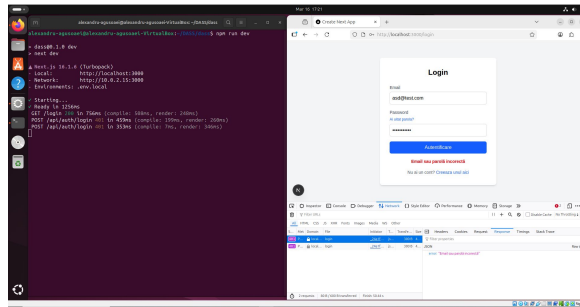


id	email	password_hash	role	created_at
649565f3-b668-4d5a-b666-dde4c94b7f1	agu@gmail.com	\$2b\$10\$F1GxYfmVj734W9p4eDPu2y/l	USER	2026-03-16 13:39:43.502

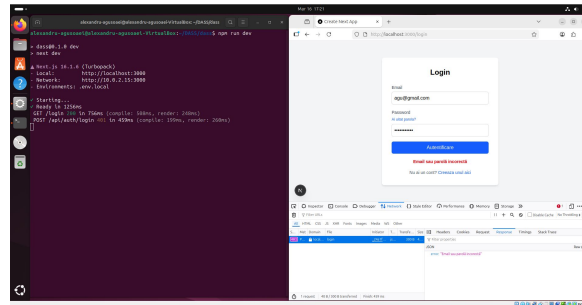
Figura 12: Parola securizată prin algoritmul `bcrypt`, vizibilă în baza de date Supabase.

8.2 Validare 4.4: User Enumeration

Acțiune: Am reluat testul trimițând cereri succesive de autentificare: una cu o adresă de email inexistentă, iar cealaltă cu o adresă validă, dar parolă incorectă.



(a) Mesaj generic la încercarea de autentificare cu un cont inexistent.



(b) Mesaj generic la încercarea de autentificare cu parola gresita.

Figura 13: Validarea remedierii: Serverul returnează răspunsuri identice, prevenind scurgerea de informații despre existența conturilor.

Rezultat: Ambele cereri au returnat exact același răspuns HTTP (401 Unauthorized) și același mesaj de eroare: „*Email sau parolă incorectă*”. Atacatorul nu mai poate distinge care conturi sunt valide.

8.3 Validare 4.3: Protecție Brute Force (Rate Limiting)

Acțiune: Am rulat din nou scriptul automat `brute-force.js` creat anterior, care trimite cereri multiple de autentificare într-un interval scurt de timp.

Rezultat: Primele 5 cereri au primit statusul 401 (parolă greșită), dar începând cu a 6-a încercare, mecanismul de limitare s-a activat. Serverul a început să returneze statusul 429 Too Many Requests și mesajul „*Prea multe încercări eșuate. Cont blocat temporar*”, mitigând complet atacul automatizat.

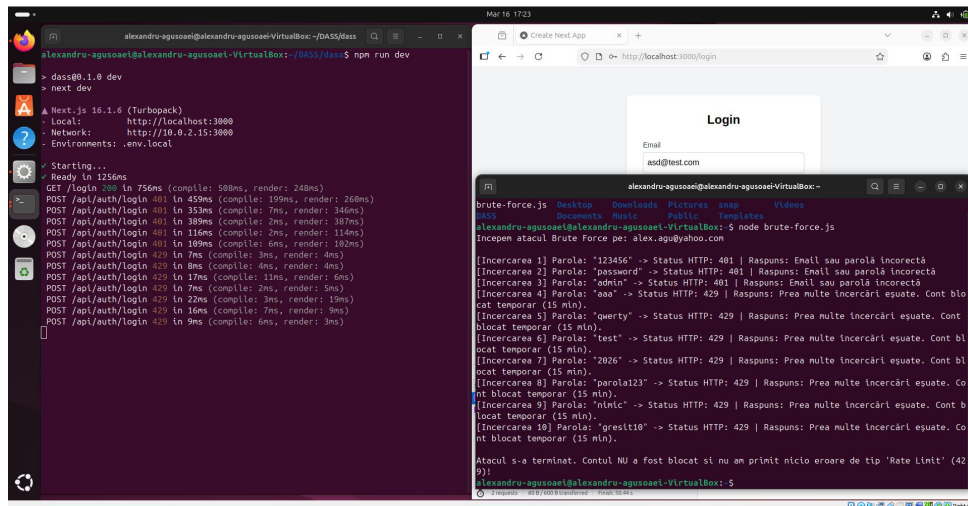


Figura 14: Rularea scriptului de Brute Force evidențiind blocarea atacatorului prin codul HTTP 429.

8.4 Validare 4.5: Securitatea Sesiunilor

Acțiune: M-am autentificat cu succes și am inspectat cookie-ul de sesiune din *Developer Tools*.

Rezultat: Flag-ul `HttpOnly` este acum prezent și activat, împiedicând accesarea token-ului prin comanda `document.cookie` din consolă. Această setare neutralizează riscul de furt al sesiunii prin vulnerabilități XSS.

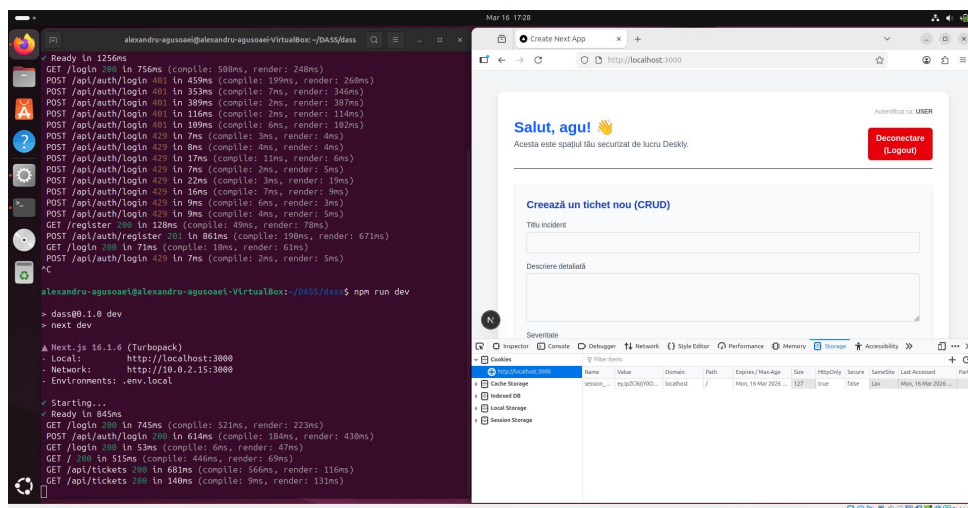


Figura 15: Cookie securizat

8.5 Validare 4.6: Resetarea Sigură a Parolei

Acțiune: Am încercat să bypass-ez mecanismul de resetare accesând URL-ul folosind vechiul payload (adresa de email encodată în Base64).

Rezultat: Cererea POST către `/api/auth/reset-password` a fost respinsă de server cu eroarea „*Token invalid sau expirat!*”. Backend-ul necesită acum o semnătură validă HMAC SHA-256 pentru JWT, demonstrând că token-urile predictibile nu mai funcționează.

Figura 16: Sistemul respinge atacul de tip predictibil, solicitând un token JWT valid.

9 Audit și Jurnalizare (Logging & Monitoring)

Implementarea unor mecanisme robuste de securitate preventivă (precum Rate Limiting sau Hashing) este incompletă fără un sistem de monitorizare care să asigure trasabilitatea și non-repudierea acțiunilor. Pentru a satisface această cerință, aplicația Deskly utilizează tabelul `audit_logs` din baza de date.

9.1 Trasabilitatea Evenimentelor de Securitate

Orice acțiune critică efectuată în sistem este înregistrată automat. Jurnalizarea acoperă următoarele elemente esențiale:

- **Cine:** Identificatorul utilizatorului (`user_id`) sau adresa de email încercată.
- **Ce:** Tipul acțiunii (ex: `LOGIN_SUCCESS`, `LOGIN_FAILED`, `TICKET_CREATED`, `PASSWORD_RESET`).
- **De unde:** Adresa IP a utilizatorului (`ip_address`), esențială pentru corelarea atacurilor distribuite.
- **Când:** Marcajul de timp (`created_at`), generat automat și inalterabil de către serverul de baze de date PostgreSQL.

9.2 Răspunsul la Incidente (Incident Response)

Existența acestor loguri ne-a permis să detectăm retrospectiv atacul de tip Brute Force demonstrat în secțiunea de PoC. Analizând tabelul `audit_logs`, un administrator de securitate (SOC Analyst) poate observa zeci de evenimente `LOGIN_FAILED` generate în decurs de câteva secunde de la aceeași adresă IP. Mai mult, trasabilitatea la nivel de business previne alterarea frauduloasă a datelor: dacă un utilizator malițios modifică sau șterge un tichet, acțiunea sa este definitiv înregistrată, asigurând integritatea proceselor interne ale companiei AuthX.

id	user_id	action	resource	text
1b806365-5c44-4b7d-b7f1-9c8c1a6de517	b1332806-0781-410e-9a44-db32f...	TICKET_SEARCH	Query: asd	NULL
37e1ead0-c81f-4d84-96f9-15a354c1dc35	649565f3-b668-4daa-ba66-d6a5...	TICKET_SEARCH	Query: ser	NULL
3e70e650-6a24-42dd-99d9-a295cdeb104	649565f3-b668-4daa-ba66-d6a5...	TICKET_CREATED	Ticket ID: 243ea896-c6da-4f7f-9f78-05a9	NULL
3fa3c92d-f39a-414a-83aa-c0f9a9a2dd85	649565f3-b668-4daa-ba66-d6a5...	TICKET_CREATED	Ticket ID: 211ectef-31e3-4353-a245-14868	NULL
7b865fbb-b442-434f-88df-1b7c99b-3a2e	b1332806-0781-410e-9a44-db32f...	LOGIN_SUCCESS	Autentificare reușită	NULL
8a77030f-bfab-4194-bcdf-5320337412a6	649565f3-b668-4daa-ba66-d6a5...	TICKET_SEARCH	Query: seef	NULL
927e886e-74fd-46b5-8c8d-36344d9b9c9a	b1332806-0781-410e-9a44-db32f...	LOGIN_FAILED	Email Incercat: alex@yahoo.com	NULL
b0eeda27-b605-48d6-8009-ca150946947	b1332806-0781-410e-9a44-db32f...	LOGIN_FAILED	Email Incercat: alex@yahoo.com	NULL
b80a336f-063e-433d-9e59-19eb5701a031	b1332806-0781-410e-9a44-db32f...	LOGIN_SUCCESS	Autentificare reușită	NULL
d99b758-1416-40d4-96d4-77cbf4eeab6	b1332806-0781-410e-9a44-db32f...	PASSWORD_RESET_SUCCESS	Parola a fost schimbată pentru alex@yahc	NULL

resource	resource_id	timestamp	ip_address
Query: asd	NULL	2026-03-17 10:15:12.042235+0	::ffff:127.0.0.1
Query: ser	NULL	2026-03-16 14:06:13.608812+0	:::1
Ticket ID: 243ea896-c6da-4f7f-9f78-05a9	NULL	2026-03-16 14:05:56.520372+0	:::1
Ticket ID: 211ectef-31e3-4353-a245-14868	NULL	2026-03-16 14:06:09.811433+0	:::1
Autentificare reușită	NULL	2026-03-17 10:48:28.875463+0	::ffff:127.0.0.1
Query: seef	NULL	2026-03-16 14:06:18.103715+0	:::1
Email Incercat: alex@yahoo.com	NULL	2026-03-17 10:47:05.773878+0	::ffff:127.0.0.1
Email Incercat: alex@yahoo.com	NULL	2026-03-17 10:47:24.945305+0	::ffff:127.0.0.1
Autentificare reușită	NULL	2026-03-17 10:47:33.092746+0	::ffff:127.0.0.1
Parola a fost schimbată pentru alex@yahc	NULL	2026-03-17 10:48:15.367981+0	::ffff:127.0.0.1

Figura 17: Vizualizarea tabelului de audit evidențiind înregistrarea tentativelor de autentificare eșuate și a acțiunilor utilizatorilor.

10 Concluzii și Lecții Învățate

Dezvoltarea, auditarea și securizarea aplicației a reprezentat o incursiune completă în ciclul de viață al dezvoltării software securizate (Secure SDLC). Trecerea prin rolurile duale de *Security Tester* și *Developer* a demonstrat în mod practic faptul că securitatea nu este o funcționalitate adăugată la final, ci un proces continuu care trebuie integrat încă din faza de design (*Secure by Design*).

Principalele lecții învățate în urma acestui proiect sunt:

- **Funcționalitatea nu înseamnă Securitate:** Versiunea inițială (MVP) a aplicației funcționa perfect din perspectiva utilizatorului, dar ascundea vulnerabilități critice.

Un simplu formular de login poate deveni poarta principală de intrare pentru un atacator dacă nu este protejat de mecanisme precum Rate Limiting și politici stricte de parole.

- **Nu avea niciodată încredere în inputul clientului:** Orice date primite de la utilizator (inclusiv token-uri sau cookie-uri) pot fi manipulate. Validarea trebuie să aibă loc întotdeauna pe server (Backend), iar utilizarea standardelor criptografice consacrate (cum ar fi bcrypt pentru parole și JWT semnate pentru token-uri) este non-negociabilă.
- **Importanța vizibilității (Logging):** Fără un tabel precum `audit_logs`, un atac de tip Brute Force sau o scurgere de date ar fi rămas complet nedetectate, lăsând organizația fără nicio posibilitate de răspuns la incident (Incident Response).
- **Complexitatea atacurilor moderne:** Lanțurile de atac (Attack Chaining) sunt mult mai periculoase decât vulnerabilitățile izolate. Combinarea Enumerării Utilizatorilor cu o lipsă de Rate Limiting și o stocare nesigură a parolelor duce inevitabil la o compromitere totală a sistemului.

În concluzie, implementarea controalelor de securitate în versiunea **secure-v2** a redus semnificativ suprafața de atac (Attack Surface) a aplicației, transformând-o dintr-un sistem vulnerabil într-o platformă rezilientă, pregătită să facă față amenințărilor din mediul real de producție.